

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №3

«Модульне тестування програмного коду (Unit Testing)»

з дисципліни **«Основи програмної інженерії»**

Виконав:

ст. гр. ПЗП-25-5

Козирь В.С.

Прийняв:

Гребенюк В. О

Харків 2026

3 МОДУЛЬНЕ ТЕСТУВАННЯ ПРОГРАМНОГО КОДУ (UNIT TESTING)

3.1 Мета роботи

Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів - еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80 %.

GitHub: <https://github.com/Mykhailo-Berkuta/2Win/tree/feature/Kozyr>

Робота виконана на основі UML діаграм з Лабораторної роботи 2

Під час роботи було виконано «Бонусне» завдання.

3.2 Вихідний код

```
class AuthService {
  constructor() {
    this.users = [];
  }

  /**
   * Реєструє нового користувача.
   * EP: валідний email/пароль/isAdult → User; невалідні → Error
   * BVA: мінімальна довжина пароля = 6 символів
   */
  register(email, pwd, isAdult) {
    if (!email || typeof email !== 'string' || !email.includes('@')) {
      throw new Error('Invalid email format');
    }
    if (!pwd || typeof pwd !== 'string' || pwd.length < 6) {
      throw new Error('Password must be at least 6 characters');
    }
    if (!isAdult) {
      throw new Error('User must be an adult');
    }
    if (this.users.find(u => u.email === email)) {
      throw new Error('Email already registered');
    }

    const user = {
      userId: this.users.length + 1,
      email,
      passwordHash: Buffer.from(pwd).toString('base64'),
      isAdult,
      createdAt: new Date(),
    };
    this.users.push(user);
    return user;
  }
}
```

```

/**
 * Аутентифікує користувача.
 * EP: правильні дані → true; неправильний пароль → false; не існує → false
 */
authenticate(email, pwd) {
  if (!email || !pwd) return false;
  const user = this.users.find(u => u.email === email);
  if (!user) return false;
  return user.passwordHash === Buffer.from(pwd).toString('base64');
}

/**
 * Перевіряє підтвердження повноліття.
 * EP: true → true; false/null/undefined → false
 */
validateAge(confirmed) {
  if (confirmed === null || confirmed === undefined) return false;
  return confirmed === true;
}
}

class PredictionResult {
  constructor({ predictedOutcome, confidence, homeWinProb, drawProb, awayWinProb }) {
    this.predictedOutcome = predictedOutcome;
    this.confidence = confidence;
    this.homeWinProb = homeWinProb;
    this.drawProb = drawProb;
    this.awayWinProb = awayWinProb;
  }

  getSummary() {
    return `Outcome: ${this.predictedOutcome} (confidence: ${(this.confidence *
100).toFixed(1)}%)`;
  }
}

class PredictionService {
  constructor(modelVersion = 'v1.0') {
    this.modelVersion = modelVersion;
  }

  /**
   * Розраховує імовірності результатів матчу.
   * EP: match.status === 'upcoming' → PredictionResult
   *      match.status === 'finished' → Error
   *      match без рейтингів → рівномірний розподіл
   * BVA: homeRating та awayRating від 0 до 100
   */
  calculateProbability(match) {
    if (!match || typeof match !== 'object') {
      throw new Error('Match object is required');
    }
    if (match.status !== 'upcoming') {

```

```

    throw new Error('Prediction unavailable: match is not upcoming');
  }

  const home = match.homeRating ?? 50;
  const away = match.awayRating ?? 50;

  if (home < 0 || home > 100 || away < 0 || away > 100) {
    throw new Error('Ratings must be between 0 and 100');
  }

  const total = home + away + 20; // +20 for draw weight
  const homeWinProb = parseFloat((home / total).toFixed(4));
  const awayWinProb = parseFloat((away / total).toFixed(4));
  const drawProb = parseFloat((1 - homeWinProb - awayWinProb).toFixed(4));

  let predictedOutcome;
  if (homeWinProb > awayWinProb && homeWinProb > drawProb) {
    predictedOutcome = 'home_win';
  } else if (awayWinProb > homeWinProb && awayWinProb > drawProb) {
    predictedOutcome = 'away_win';
  } else {
    predictedOutcome = 'draw';
  }

  return new PredictionResult({
    predictedOutcome,
    confidence: Math.max(homeWinProb, drawProb, awayWinProb),
    homeWinProb,
    drawProb,
    awayWinProb,
  });
}

/**
 * Повертає список коефіцієнтів від букмекерів.
 * EP: match з odds → List<OddsEntry>; match без odds → []
 * BVA: odds.length === 0 → порожній масив
 */
fetchOdds(match) {
  if (!match || typeof match !== 'object') {
    throw new Error('Match object is required');
  }
  if (!Array.isArray(match.odds)) return [];

  return match.odds.map(entry => ({
    bookmaker: entry.bookmaker,
    homeOdds: entry.homeOdds,
    drawOdds: entry.drawOdds,
    awayOdds: entry.awayOdds,
  }));
}

module.exports = { AuthService, PredictionService, PredictionResult };

```

3.3 Таблиця проєктування тестів

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
ТС-01: реєструє користувача з валідними даними	email='test@example.com', pwd='password123', isAdult=true	User об'єкт з userId=1	ЕР (позитивний)	pass
ТС-02: кидає Error при неправильно му форматі email	email='invalidemail', pwd='password123', isAdult=true	Error: 'Invalid email format'	ЕР (негативний)	pass
ТС-03: кидає Error при порожньому email	email='', pwd='password123', isAdult=true	Error: 'Invalid email format'	ЕР (негативний)	pass
ТС-04: кидає Error при паролі довжиною 5 символів	email='a@b.com', pwd='12345', isAdult=true	Error: 'Password must be at least 6 characters'	BVA (межа-1)	pass
ТС-05: реєструє при паролі рівно 6 символів	email='a@b.com', pwd='123456', isAdult=true	User об'єкт	BVA (точна межа)	pass
ТС-06: кидає Error якщо isAdult = false	email='adult@b.com', pwd='password', isAdult=false	Error: 'User must be an adult'	ЕР (негативний)	pass
ТС-07: кидає Error при	email='dup@b.com' (вже існує),	Error: 'Email already	ЕР (негативний)	pass

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
повторний реєстрації	pwd='pass123', isAdult=true	registered'	ий)	
ТС-08: повертає true при правильних credentials	email='user@b.com', pwd='mypassword' (вірний)	true	ЕР (позитивний)	pass
ТС-09: повертає false при невірному паролі	email='user@b.com', pwd='wrongpass'	false	ЕР (негативний)	pass
ТС-10: повертає false якщо email не існує	email='ghost@b.com', pwd='any'	false	ЕР (негативний)	pass
ТС-11: повертає false при порожніх аргументах	email="", pwd=""	false	BVA (порожній рядок)	pass
ТС-12: повертає true при confirmed = true	confirmed=true	true	ЕР (позитивний)	pass
ТС-13: повертає false при confirmed = false	confirmed=false	false	ЕР (негативний)	pass

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
ТС-14: повертає false при confirmed = null	confirmed=null	false	BVA (граничне nullish)	pass
ТС-15: повертає PredictionResult для upcoming матчу	status='upcoming', homeRating=70, awayRating=50	PredictionResult, сума ймовірностей ≈ 1	ЕР (позитивний)	pass
ТС-16: кидає Error для finished матчу	status='finished', homeRating=60, awayRating=40	Error: 'Prediction unavailable'	ЕР (негативний)	pass
ТС-17: кидає Error якщо рейтинг < 0	status='upcoming', homeRating=-1, awayRating=50	Error: 'Ratings must be between 0 and 100'	BVA (межа-1)	pass
ТС-18: кидає Error якщо рейтинг > 100	status='upcoming', homeRating=50, awayRating=101	Error: 'Ratings must be between 0 and 100'	BVA (межа+1)	pass
ТС-19: predict=home_win коли homeRating >> awayRating	status='upcoming', homeRating=100, awayRating=0	predictedOutcome='home_win'	ЕР (позитивний)	pass
ТС-20: predict=away_win коли awayRating	status='upcoming', homeRating=0, awayRating=100	predictedOutcome='away_win'	ЕР (позитивний)	pass

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
>> homeRating				
ТС-21: використовує дефолтний рейтинг 50 якщо не вказано	status='upcoming', рейтинги відсутні	homeWinProb === awayWinProb	ЕР (позитивний)	pass
ТС-22: кидає Error якщо match не є об'єктом	match=null	Error: 'Match object is required'	ЕР (негативний)	pass
ТС-23: повертає масив OddsEntry для матчу з odds	odds=[{bookmaker:'Bet365', homeOdds:1.5, drawOdds:3.2, awayOdds:4.0}]	масив з 1 елементом, bookmaker='Bet365'	ЕР (позитивний)	pass
ТС-24: повертає порожній масив якщо odds=[]	odds=[]	[]	BVA (length=0)	pass
ТС-25: повертає порожній масив якщо поле odds відсутнє	match без поля odds	[]	ЕР (негативний)	pass
ТС-26: повертає	predictedOutcome='hom	рядок містить	ЕР (позитивний)	pass

Тест-кейс	Вхідні дані	Очікуваний результат	Техніка	Статус
форматований рядок з результатом	e_win', confidence=0.65	'home_win' та '65.0%'	ий)	
ТС-27: кидає Error якщо match=undefined у fetchOdds	match=undefined	Error: 'Match object is required'	ЕР (негативний)	pass

Рис 3.1 - Таблиця проектування тестів

3.4 Вихідний код тестового набору з коментарями

```
const {
  AuthService,
  PredictionService,
  PredictionResult,
} = require("../src/predictionService");

// _____
// AuthService - register()
// _____

describe("AuthService.register()", () => {
  test("ТС-01: реєструє користувача з валідними даними", () => {
    // Technique: ЕР - позитивний клас (валідні email, пароль, isAdult=true)
    // Arrange
    const auth = new AuthService();
    // Act
    const user = auth.register("test@example.com", "password123", true);
    // Assert
    expect(user.email).toBe("test@example.com");
    expect(user.isAdult).toBe(true);
    expect(user.userId).toBe(1);
  });

  test("ТС-02: кидає Error при неправильному форматі email", () => {
    // Technique: ЕР - негативний клас (email без @)
    // Arrange
    const auth = new AuthService();
    // Act & Assert
```

```

    expect(() => auth.register("invalidemail", "password123", true)).toThrow(
      "Invalid email format",
    );
  });

test("TC-03: кидає Error при порожньому email", () => {
  // Technique: EP - негативний клас (email = '')
  // Arrange
  const auth = new AuthService();
  // Act & Assert
  expect(() => auth.register("", "password123", true)).toThrow(
    "Invalid email format",
  );
});

test("TC-04: кидає Error при паролі довжиною 5 символів (BVA: межа -1)", () => {
  // Technique: BVA - межа мін. довжини пароля (6), значення 5
  // Arrange
  const auth = new AuthService();
  // Act & Assert
  expect(() => auth.register("a@b.com", "12345", true)).toThrow(
    "Password must be at least 6 characters",
  );
});

test("TC-05: реєструє при паролі рівно 6 символів (BVA: точна межа)", () => {
  // Technique: BVA - мінімально допустима довжина пароля = 6
  // Arrange
  const auth = new AuthService();
  // Act
  const user = auth.register("a@b.com", "123456", true);
  // Assert
  expect(user).toBeDefined();
});

test("TC-06: кидає Error якщо isAdult = false", () => {
  // Technique: EP - негативний клас (неповнолітній)
  // Arrange
  const auth = new AuthService();
  // Act & Assert
  expect(() => auth.register("adult@b.com", "password", false)).toThrow(
    "User must be an adult",
  );
});

test("TC-07: кидає Error при повторній реєстрації з тим самим email", () => {
  // Technique: EP - негативний клас (дублікат email)
  // Arrange
  const auth = new AuthService();
  auth.register("dup@b.com", "pass123", true);
  // Act & Assert
  expect(() => auth.register("dup@b.com", "pass123", true)).toThrow(
    "Email already registered",
  );
});

```

```

});
});

// -----
// AuthService - authenticate() та validateAge()
// -----

describe("AuthService.authenticate()", () => {
  test("TC-08: повертає true при правильних credentials", () => {
    // Technique: EP - позитивний клас (існуючий user, вірний пароль)
    // Arrange
    const auth = new AuthService();
    auth.register("user@b.com", "mypassword", true);
    // Act
    const result = auth.authenticate("user@b.com", "mypassword");
    // Assert
    expect(result).toBe(true);
  });

  test("TC-09: повертає false при невірному паролі", () => {
    // Technique: EP - негативний клас (неправильний пароль)
    // Arrange
    const auth = new AuthService();
    auth.register("user@b.com", "mypassword", true);
    // Act
    const result = auth.authenticate("user@b.com", "wrongpass");
    // Assert
    expect(result).toBe(false);
  });

  test("TC-10: повертає false якщо email не існує", () => {
    // Technique: EP - негативний клас (незареєстрований email)
    // Arrange
    const auth = new AuthService();
    // Act
    const result = auth.authenticate("ghost@b.com", "any");
    // Assert
    expect(result).toBe(false);
  });

  test("TC-11: повертає false при порожніх аргументах", () => {
    // Technique: BVA - порожній рядок (межа валідного рядка)
    // Arrange
    const auth = new AuthService();
    // Act & Assert
    expect(auth.authenticate("", "")).toBe(false);
  });
});

describe("AuthService.validateAge()", () => {
  test("TC-12: повертає true при confirmed = true", () => {
    // Technique: EP - позитивний клас
    // Arrange
    const auth = new AuthService();
  });
});

```

```

    // Act & Assert
    expect(auth.validateAge(true)).toBe(true);
  });

  test("TC-13: повертає false при confirmed = false", () => {
    // Technique: EP - негативний клас
    // Arrange
    const auth = new AuthService();
    // Act & Assert
    expect(auth.validateAge(false)).toBe(false);
  });

  test("TC-14: повертає false при confirmed = null (BVA: граничне nullish значення)", () => {
    // Technique: BVA - null як граничний nullish
    // Arrange
    const auth = new AuthService();
    // Act & Assert
    expect(auth.validateAge(null)).toBe(false);
  });
});

// -----
// PredictionService - calculateProbability()
// -----

describe("PredictionService.calculateProbability()", () => {
  test("TC-15: повертає PredictionResult для upcoming матчу", () => {
    // Technique: EP - позитивний клас (status=upcoming, валідні рейтинги)
    // Arrange
    const ps = new PredictionService();
    const match = { status: "upcoming", homeRating: 70, awayRating: 50 };
    // Act
    const result = ps.calculateProbability(match);
    // Assert
    expect(result).toBeInstanceOf(PredictionResult);
    expect(
      result.homeWinProb + result.drawProb + result.awayWinProb,
    ).toBeCloseTo(1, 2);
  });

  test("TC-16: кидає Error для finished матчу", () => {
    // Technique: EP - негативний клас (status=finished)
    // Arrange
    const ps = new PredictionService();
    const match = { status: "finished", homeRating: 60, awayRating: 40 };
    // Act & Assert
    expect(() => ps.calculateProbability(match)).toThrow(
      "Prediction unavailable",
    );
  });

  test("TC-17: кидає Error якщо рейтинг < 0 (BVA: межа 0, значення -1)", () => {
    // Technique: BVA - рейтинг нижче мінімуму
    // Arrange

```

```

const ps = new PredictionService();
const match = { status: "upcoming", homeRating: -1, awayRating: 50 };
// Act & Assert
expect(() => ps.calculateProbability(match)).toThrow(
  "Ratings must be between 0 and 100",
);
});

test("TC-18: кидає Error якщо рейтинг > 100 (BVA: межа 100, значення 101)", () => {
  // Technique: BVA - рейтинг вище максимуму
  // Arrange
  const ps = new PredictionService();
  const match = { status: "upcoming", homeRating: 50, awayRating: 101 };
  // Act & Assert
  expect(() => ps.calculateProbability(match)).toThrow(
    "Ratings must be between 0 and 100",
  );
});

test("TC-19: predict=home_win коли homeRating >> awayRating", () => {
  // Technique: EP - позитивний клас (явна перевага home)
  // Arrange
  const ps = new PredictionService();
  const match = { status: "upcoming", homeRating: 100, awayRating: 0 };
  // Act
  const result = ps.calculateProbability(match);
  // Assert
  expect(result.predictedOutcome).toBe("home_win");
});

test("TC-20: predict=away_win коли awayRating >> homeRating", () => {
  // Technique: EP - позитивний клас (явна перевага away)
  // Arrange
  const ps = new PredictionService();
  const match = { status: "upcoming", homeRating: 0, awayRating: 100 };
  // Act
  const result = ps.calculateProbability(match);
  // Assert
  expect(result.predictedOutcome).toBe("away_win");
});

test("TC-21: використовує дефолтний рейтинг 50 якщо не вказано", () => {
  // Technique: EP - позитивний клас (рейтинги відсутні → дефолт)
  // Arrange
  const ps = new PredictionService();
  const match = { status: "upcoming" };
  // Act
  const result = ps.calculateProbability(match);
  // Assert
  expect(result.homeWinProb).toBe(result.awayWinProb);
});

test("TC-22: кидає Error якщо match не є об'єктом", () => {
  // Technique: EP - негативний клас (null-значення)

```

```

    // Arrange
    const ps = new PredictionService();
    // Act & Assert
    expect(() => ps.calculateProbability(null)).toThrow(
        "Match object is required",
    );
});

});

// -----
// PredictionService - fetchOdds()
// -----

describe("PredictionService.fetchOdds()", () => {
    test("TC-23: повертає масив OddsEntry для матчу з odds", () => {
        // Technique: EP - позитивний клас (match з odds масивом)
        // Arrange
        const ps = new PredictionService();
        const match = {
            status: "upcoming",
            odds: [
                { bookmaker: "Bet365", homeOdds: 1.5, drawOdds: 3.2, awayOdds: 4.0 },
            ],
        };
        // Act
        const result = ps.fetchOdds(match);
        // Assert
        expect(result).toHaveLength(1);
        expect(result[0].bookmaker).toBe("Bet365");
    });

    test("TC-24: повертає порожній масив якщо odds відсутні (BVA: length=0)", () => {
        // Technique: BVA - порожній масив odds
        // Arrange
        const ps = new PredictionService();
        const match = { status: "upcoming", odds: [] };
        // Act
        const result = ps.fetchOdds(match);
        // Assert
        expect(result).toEqual([]);
    });

    test("TC-25: повертає порожній масив якщо поле odds відсутнє", () => {
        // Technique: EP - негативний клас (match без поля odds)
        // Arrange
        const ps = new PredictionService();
        const match = { status: "upcoming" };
        // Act
        const result = ps.fetchOdds(match);
        // Assert
        expect(result).toEqual([]);
    });

    test("TC-26: кидає Error якщо match=undefined у fetchOdds", () => {

```

```

// Technique: EP - негативний клас (undefined замість об'єкта)
// Arrange
const ps = new PredictionService();
// Act & Assert
expect(() => ps.fetchOdds(undefined)).toThrow("Match object is required");
});
});

// _____
// PredictionResult - getSummary()
// _____

describe("PredictionResult.getSummary()", () => {
  test("TC-27: повертає форматований рядок з результатом", () => {
    // Technique: EP - позитивний клас (валідні дані)
    // Arrange
    const result = new PredictionResult({
      predictedOutcome: "home_win",
      confidence: 0.65,
      homeWinProb: 0.65,
      drawProb: 0.2,
      awayWinProb: 0.15,
    });
    // Act
    const summary = result.getSummary();
    // Assert
    expect(summary).toContain("home_win");
    expect(summary).toContain("65.0%");
  });
});
});

```

3.5 Скріншот та HTML-звіт покриття коду з відображенням відсотку line coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
predictionService.js	100	100	100	100	
Test Suites: 1 passed, 1 total Tests: 27 passed, 27 total Snapshots: 0 total Time: 0.629 s, estimated 1 s Ran all test suites.					

Рис 3.2 – Скріншот терміналу тестів

All files

100% Statements 55/55 100% Branches 55/55 100% Functions 12/12 100% Lines 48/48

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
predictionService.js		100%	55/55	100%	55/55
				100%	12/12
					100%
					48/48

Рис 3.3 - HTML-звіт покриття коду

3.6 Посилання на Git-репозиторій з кодом модуля та тестами.

<https://github.com/Mykhailo-Berkuta/2Win/tree/feature/Kozyr>

3.7 Висновки

У ході виконання лабораторної роботи 3 було набуто практичних навичок із модульного тестування програмного коду з використанням фреймворку Jest для мови JavaScript.

На основі UML-моделі, розробленої в лабораторній роботі 2, було реалізовано програмний модуль predictionService.js, що містить три класи -

AuthService, PredictionService та PredictionResult - з нетривіальною логікою: умовними конструкціями, циклами та обробкою виняткових ситуацій. Модуль відповідає діаграмі класів і покриває функціональні вимоги KGLG-01 та KGLG-02 системи прогнозування ставок на спорт.

Для проєктування тестів було застосовано дві формальні техніки:

- Equivalence Partitioning (EP) - вхідний простір поділено на класи допустимих та недопустимих значень, з кожного обрано по одному представнику
- Boundary Value Analysis (BVA) - протестовано граничні значення параметрів: мінімальна довжина пароля (5/6 символів), рейтинги команд (-1/0 та 100/101), порожні рядки та null-значення

Було написано 27 модульних тестів, структурованих за патерном AAA (Arrange - Act - Assert), кожен із коментарем про застосовану техніку. Усі 27 тестів успішно пройшли виконання.

Додатково налаштовано автоматичний запуск тестів через GitHub Actions - при кожному push або pull request до гілки main виконується встановлення залежностей, запуск тестів та збереження HTML-звіту покриття як артефакту.

Виявлені проблеми

У процесі виконання було виявлено одну проблему: після першого запуску тестів рядок 130 (`predictionService.js`) залишався непокритим - це була гілка перевірки `match=undefined` у методі `fetchOdds()`. Причина полягала в тому, що початковий набір тестів покривав лише `null`, але не `undefined` як окремий граничний випадок JavaScript. Проблема вирішено додаванням тест-кейсу TC-27, після чого покриття досягло 100%.

Шляхи поліпшення

Якість тестів:

- Додати тести на мутаційне тестування (mutation testing) за допомогою Stryker - 100% line coverage не гарантує відсутності логічних помилок, оскільки не перевіряє всі можливі комбінації значень

Якість модуля:

- Замінити `Buffer.from(pwd).toString('base64')` на повноцінне хешування через `bcrypt` або `crypto` - поточна реалізація є лише навчальним спрощенням і не придатна для продакшену
- Додати валідацію формату email через регулярний вираз замість перевірки лише наявності символу `@`, що дозволить відхиляти некоректні адреси на кшталт `a@` або `@b`
- Реалізувати метод `calculateProbability()` з урахуванням більшої кількості факторів (форма команди, результати попередніх матчів), що зробить логіку реалістичнішою

Автоматизація:

- Налаштувати badge покриття у README.md через Codecov або Coveralls для візуального відображення поточного стану тестів у репозиторії
- Додати до GitHub Actions перевірку лінтером (ESLint) як окремий крок пайплайну для контролю якості коду